

```
public class AList {
    private int[] items;
    private int size;

    public AList() {
        items = new int[999];
        size = 0;
    }

    public void addLast(int x) {
        items[size] = x;
        size += 1;
    }

    public int get(int i) {
        return items[i];
    }

    public int size() {
        return size;
    }
}
```

What are some advantages and disadvantages of our AList? Was there anything better about DLList?

```
public class DLList<Pizza> {
    private Node sentinel;
    private int size;

    public class Node {
        public Pizza item;
        public Node prev;
        public Node next;
        ...
    }

    ...
}
```

Coordinators for Computer Science Mentors for 61B are here:

- CSM is a student-run organization that provides weekly small group tutoring sections to students in groups of 4-6
- Our goal is to provide mentorship to students in lower divs at Berkeley!
- Sections open now on scheduler - go to [scheduler.csmentors.org](https://scheduler.csmentors.org)
- Flyers available!



## Announcements

---

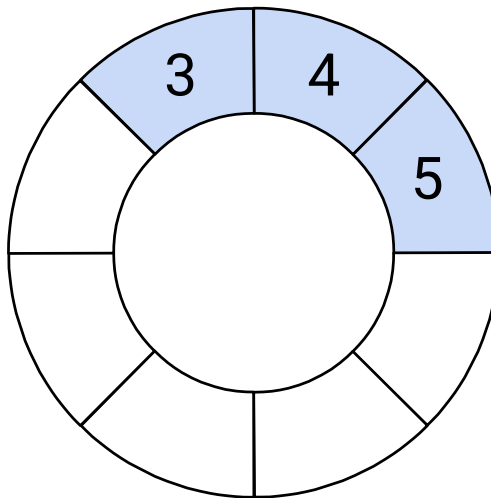
See <https://sp26.datastructur.es/>

Midterm 1: Computer Based Testing Facility during Week 6

Code reviews start next week

Computer Science Mentors (CSM) (Michelle)

[3, 4, 5]

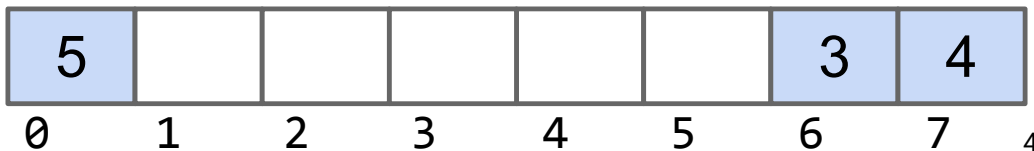


Lecture 8 (Lists 4)

# ArrayLists

CS61B, Spring 2026 @ UC Berkeley

Josh Hug and Kay Ousterhout



# AList vs. DLList

## IntList, SLList, and DLList Improvements

- |    |                                                                                              |
|----|----------------------------------------------------------------------------------------------|
| #1 | Rebranding: <code>IntList</code> → <code>IntNode</code>                                      |
| #2 | Bureaucracy: <code>SLList</code>                                                             |
| #3 | Access Control: <code>public</code> → <code>private</code>                                   |
| #4 | Nested Class: Bringing <code>IntNode</code> into <code>SLList</code>                         |
| #5 | Caching: Saving size as an int.                                                              |
| #6 | Generalizing: Adding a <code>sentinel</code> node to allow representation of the empty list. |
| #7 | Looking back: <code>.last</code> and <code>.prev</code> allow fast <code>removeLast</code>   |
| #8 | Sentinel upgrade: Avoiding special cases with <code>sentBack</code> or circular list.        |
| #9 | Generics: DLList can store anything                                                          |

## Last time: DLList Remaining Problems

- ~~Slow to get to the end of the list!~~
- ~~Only stores ints~~
- **Slow to access items in the middle of the list**

## AList Improvements

- |    |                                           |
|----|-------------------------------------------|
| #1 | Fast to access items anywhere in the list |
|----|-------------------------------------------|

```
public class AList {
    private int[] items;
    private int size;

    public AList() {
        items = new int[999];
        size = 0;
    }

    public void addLast(int x) {
        items[size] = x;
        size += 1;
    }

    public int get(int i) {
        return items[i];
    }

    public int size() {
        return size;
    }
}
```

What are some advantages and disadvantages of our AList? Was there anything better about DLList?

- Harder to add into the middle (but in a Linked List, still need to walk to the middle)
- We allocated soooooo much memory! Wasting a lot of those 999 spaces
- Can't grow beyond 999 elements

# AList Problems to Address

```
public class AList {  
    private int[] items;  
    private int size;  
  
    public AList() {  
        items = new int[999];  
        size = 0;  
    }  
  
    public void addLast(int x) {  
        items[size] = x;  
        size += 1;  
    }  
  
    public int get(int i) {  
        return items[i];  
    }  
  
    public int size() {  
        return size;  
    }  
}
```

AList problems to address today:

- Expanding our array to support unlimited elements
- Generics: making AList able to store anything
- Optimizing AList speed

| AList Improvements |                                           |
|--------------------|-------------------------------------------|
| #1                 | Fast to access items anywhere in the list |

# Resizing Array Theory

---

Lecture 8, CS61B, Spring 2026

## Resizing ArrayLists

- **Resizing Array Theory**
- Resizing Array Implementation
- Runtime Analysis (Warmup)
- Runtime Analysis
- Better Resizing Strategy

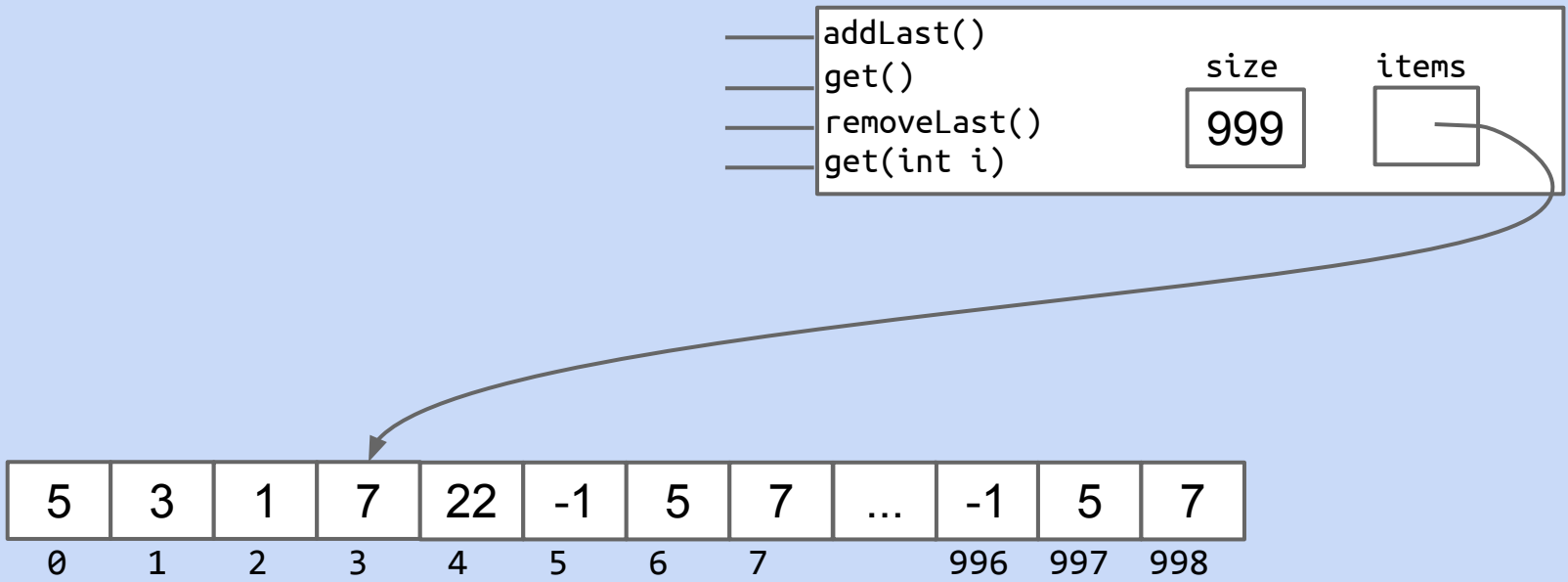
Generic ArrayLists

Optimizing ArrayLists

- Linear Implementation of Circular Arrays

# The Mighty (?) AList

Key Idea: Make an array with more capacity than we need, and addLast() uses that space.

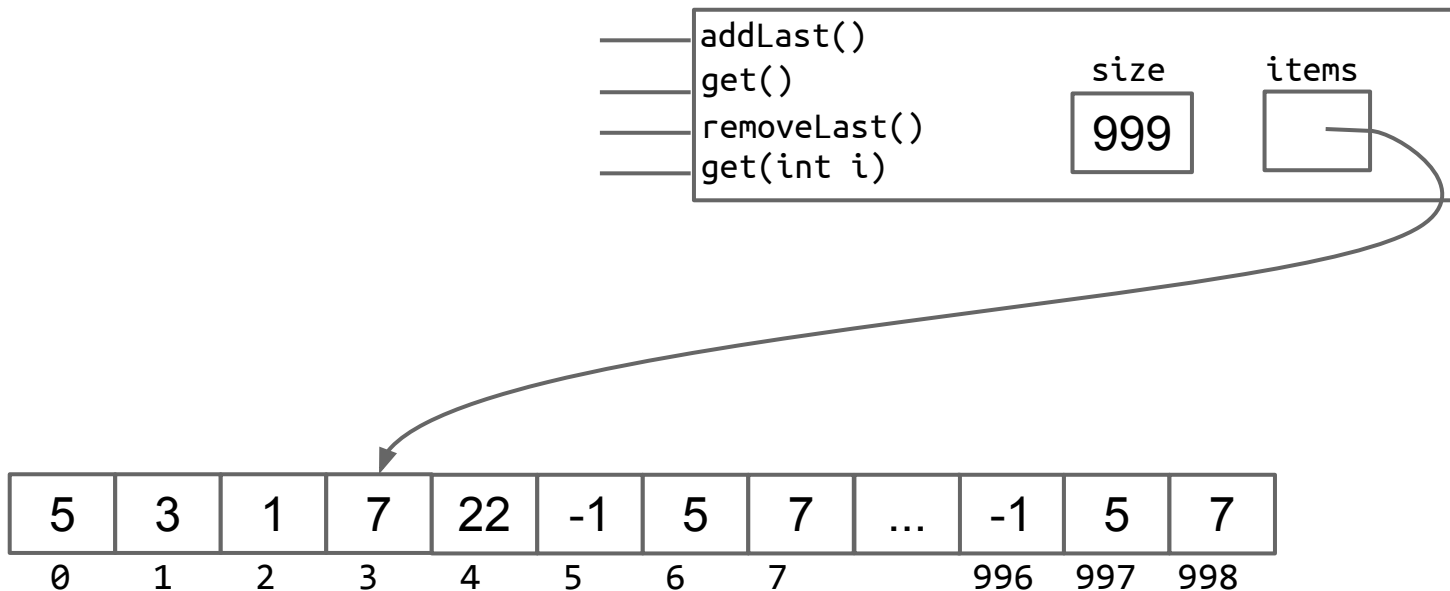


What happens if we insert into the AList above? What should we do about it?

# Array Resizing

size==items.length

When the array gets too full, e.g. `addLast(11)`, just make a new array:

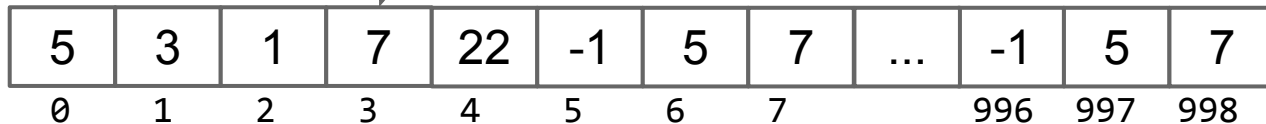


# Array Resizing

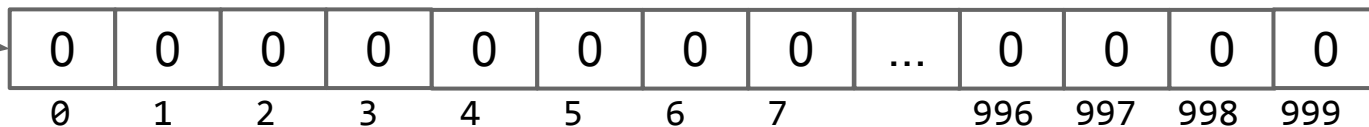
size==items.length

When the array gets too full, e.g. `addLast(11)`, just make a new array:

- `int[] resized = new int[size+1];`



resized

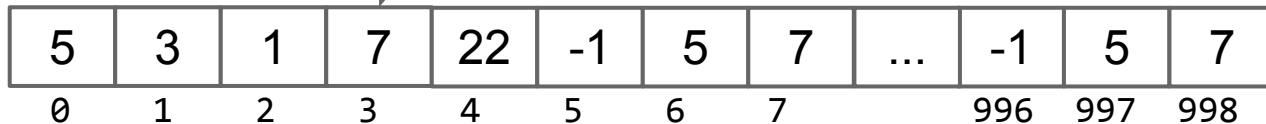


# Array Resizing

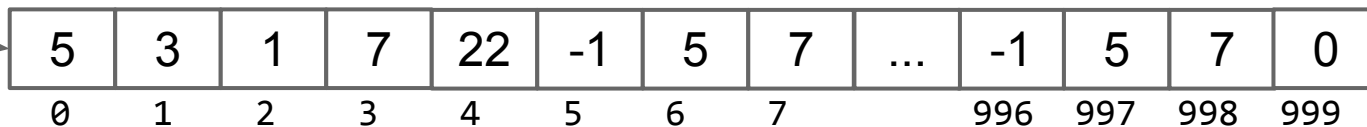
size==items.length

When the array gets too full, e.g. `addLast(11)`, just make a new array:

- `int[] resized = new int[size+1];`
- Copy over the array items



resized



# Array Resizing

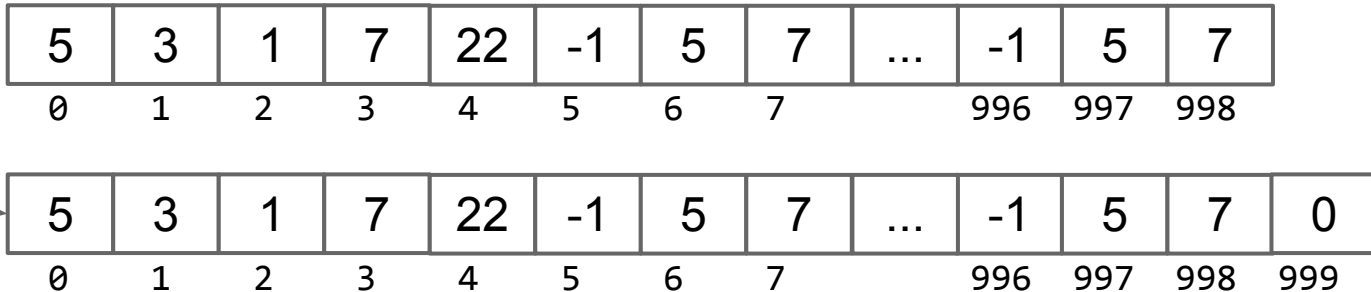
size==items.length

When the array gets too full, e.g. `addLast(11)`, just make a new array:

- `int[] resized = new int[size+1];`
- Copy over the array items
- `items = resized;`



resized



# Array Resizing

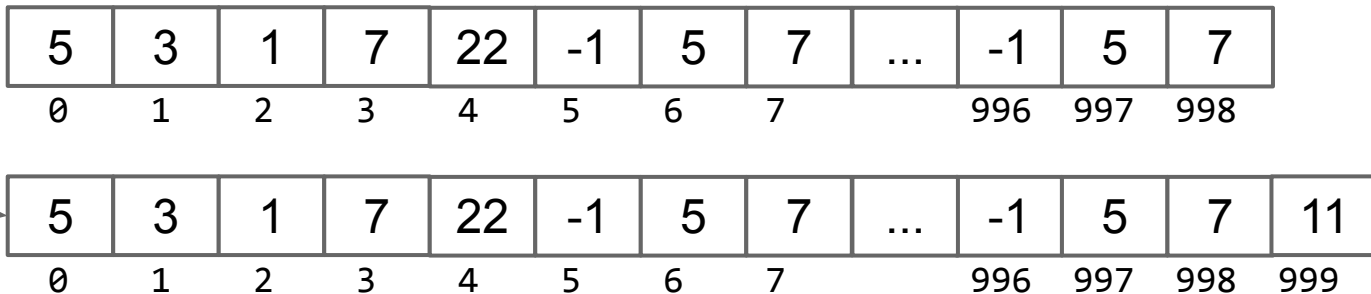
size==items.length

When the array gets too full, e.g. `addLast(11)`, just make a new array:

- `int[] resized = new int[size+1];`
- Copy over the array items
- `items = resized;`
- `items[size] = 11;`
- `size += 1;`



resized



# Array Resizing

size==items.length

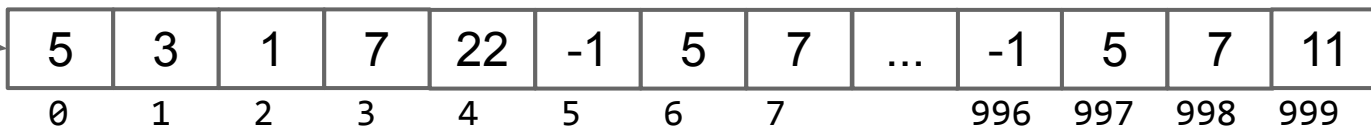
When the array gets too full, e.g. `addLast(11)`, just make a new array:

- `int[] resized = new int[size+1];`
- Copy over the array items
- `items = resized;`
- `items[size] = 11;`
- `size += 1;`

We call this process "resizing"



resized



Two ways to copy arrays:

- Item by item using a loop.
- Using arraycopy. Takes 5 parameters:
  - Source array
  - Start position in source
  - Target array
  - Start position in target
  - Number to copy

```
System.arraycopy(items, 0, a, 3, 2);
```

(In Python): `a[3:5] = items[0:2]`

arraycopy is (likely to be) faster, particularly for large arrays. More compact code.

- Code is (arguably) harder to read.
- Recommendation: Don't use on project 2 (it's a trap for reasons we'll see later today!).

# Resizing Array Implementation

---

Lecture 8, CS61B, Spring 2026

## Resizing ArrayLists

- Resizing Array Theory
- **Resizing Array Implementation**
- Runtime Analysis (Warmup)
- Runtime Analysis
- Better Resizing Strategy

Generic ArrayLists

Optimizing ArrayLists

- Linear Implementation of Circular Arrays

Implement the resizing capability in addLast

```
public class AList {  
    /** Inserts x into the back of the list. */  
    public void addLast(int x) {  
        // When the array is too full...  
        // int[] resized = new int[size+1];  
        // Copy over the array items  
        // items = resized  
        items[size] = x;  
        size += 1;  
    }  
}
```



For those of you watching online, starter code is provided in the lec8\_lists4/resizeExercise folder in the lectureCode-sp26 repo

- You'll want to write code so that the add2000Items test in AListTest works.

## Coding Demo: Resizing Array

AList.java

```
public class AList {  
    /** Resizes the underlying array to the target capacity. */  
    private void resize(int capacity) {  
        int[] resized = new int[capacity];  
        for (int i = 0; i < size; i++) {  
            resized[i] = items[i];  
        }  
        items = resized;  
    }  
  
    /** Inserts x into the back of the list. */  
    public void addLast(int x) {  
        if (size == items.length) {  
            resize(size + 1);  
        }  
        items[size] = x;  
        size += 1;  
    }  
}
```

## Resizing Array Code

```
public void addLast(int x) {
    if (size == items.length) {
        int[] resized = new int[capacity];
        for (int i = 0; i < size; i++) {
            resized[i] = items[i];
        }
        items = resized;
    }
    items[size] = x;
    size += 1;
}
```

Works

```
public void resize(int capacity) {
    int[] resized = new int[capacity];
    for (int i = 0; i < size; i++) {
        resized[i] = items[i];
    }
    items = resized;
}

public void addLast(int x) {
    if (size == items.length) {
        resize(size + 1);
    }
    items[size] = x;
    size += 1;
}
```

Much Better

Easier to read and understand.  
Easier to test the correctness of each function.

# Runtime Analysis (Warmup)

---

Lecture 8, CS61B, Spring 2026

## Resizing ArrayLists

- Resizing Array Theory
- Resizing Array Implementation
- **Runtime Analysis (Warmup)**
- Runtime Analysis
- Better Resizing Strategy

Generic ArrayLists

Optimizing ArrayLists

- Linear Implementation of Circular Arrays

Suppose we have a full array of size 999. If we call `addLast` two times, how many **total** array memory boxes will we need to create and fill (for just these 2 calls)?

- A. 0
- B. 2
- C. 1,001
- D. 2,001
- E. 1,001,000



Bonus question: What is the maximum number of array boxes that Java will track at any given time? Assume that “garbage collection” happens immediately when all references to an object are lost.

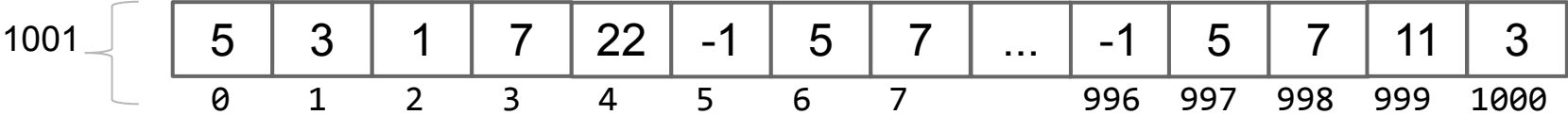
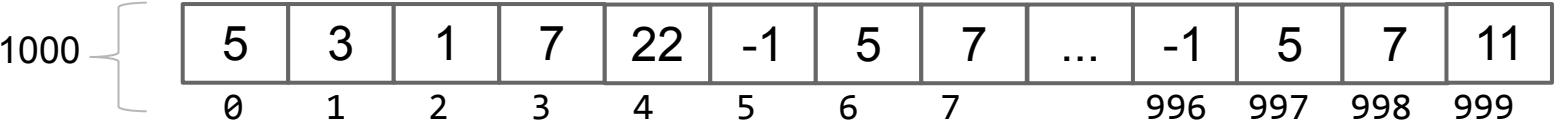
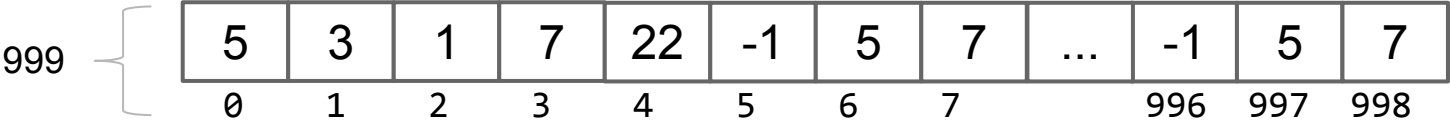
```
public void resize(int capacity) {
    int[] resized = new int[capacity];
    for (int i = 0; i < size; i++) {
        resized[i] = items[i];
    }
    items = resized;
}

public void addLast(int x) {
    if (size == items.length) {
        resize(size + 1);
    }
    items[size] = x;
    size += 1;
}
```

# Array Resizing

Resizing twice requires us to create and fill 2001 total memory boxes.

- Bonus answer: Most boxes at any one time is 2001.
- When the second addLast is done, we are left with 1001 boxes.



# Runtime Analysis

---

Lecture 8, CS61B, Spring 2026

## Resizing ArrayLists

- Resizing Array Theory
- Resizing Array Implementation
- Runtime Analysis (Warmup)
- **Runtime Analysis**
- Better Resizing Strategy

Generic ArrayLists

Optimizing ArrayLists

- Linear Implementation of Circular Arrays

## Demo: Speed Testing the ArrayList

```
jug ~/.../speedtest  
$ java SpeedTestSLList
```

```
Total runtime: 3 ms
```

SpeedTestSLList.java

```
public class SpeedTestSLList {  
    public static void main(String[] args) {  
        long startTime = System.currentTimeMillis();  
  
        SLList<Integer> L = new SLList<>();  
        int i = 0;  
        while (i < 100000) {  
            L.addFirst(i);  
            i = i + 1;  
        }  
  
        long endTime = System.currentTimeMillis();  
        System.out.println("Total runtime: "  
            + (endTime - startTime) + " ms");  
    }  
}
```

Adding 100,000 items to a new SLList is very fast (order of milliseconds).

## Demo: Speed Testing the ArrayList

```
jug ~/.../speedtest  
$ java SpeedTestAList
```

```
Total runtime: 602 ms
```

SpeedTestSLList.java

```
public class SpeedTestAList {  
    public static void main(String[] args) {  
        long startTime = System.currentTimeMillis();  
  
        AList L = new AList();  
        int i = 0;  
        while (i < 100000) {  
            L.addLast(i);  
            i = i + 1;  
        }  
  
        long endTime = System.currentTimeMillis();  
        System.out.println("Total runtime: "  
            + (endTime - startTime) + " ms");  
    }  
}
```

Adding 100,000 items to a new AList is slow (order of seconds).

Suppose we have a full array of size 2. If we call `addLast` until `size = 100`, roughly how many total array memory boxes will we need to create and fill?

- A. 100
- B. 1,000
- C. 5,000
- D. 10,000



Bonus question: What is the maximum number of array boxes that Java will track at any given time? Assume that “garbage collection” happens immediately when all references to an object are lost.

```
public void resize(int capacity) {
    int[] resized = new int[capacity];
    for (int i = 0; i < size; i++) {
        resized[i] = items[i];
    }
    items = resized;
}

public void addLast(int x) {
    if (size == items.length) {
        resize(size + 1);
    }
    items[size] = x;
    size += 1;
}
```

Suppose we have a full array of size 2. If we call addLast until size = 100, roughly how many total array memory boxes will we need to create and fill?

**C. 5,000**

Going from capacity 2 to 3: 3

From 3 to 4: 4

...

From: 99 to 100: 100

```
public void resize(int capacity) {  
    int[] resized = new int[capacity];  
    for (int i = 0; i < size; i++) {  
        resized[i] = items[i];  
    }  
    items = resized;  
}
```

We'll be doing a lot of this after the midterm.

Total array boxes created/copied:  $3 + 4 + \dots + 100$

Since sum of  $1 + 2 + 3 + \dots + N = N(N+1)/2$ ,  $\text{sum}(3, \dots, 100)$  is  $5050 - 2 - 1 = 5047$

See: <http://mathandmultimedia.com/2010/09/15/sum-first-n-positive-integers>

## Runtime and Space Usage Analysis

Since sum of  $1 + 2 + 3 + \dots + N = N(N+1)/2$ ,  $\text{sum}(3, \dots, 100)$  is close to 5050.

Rough intuition: Form pairs that all sum to  $N+1$ :

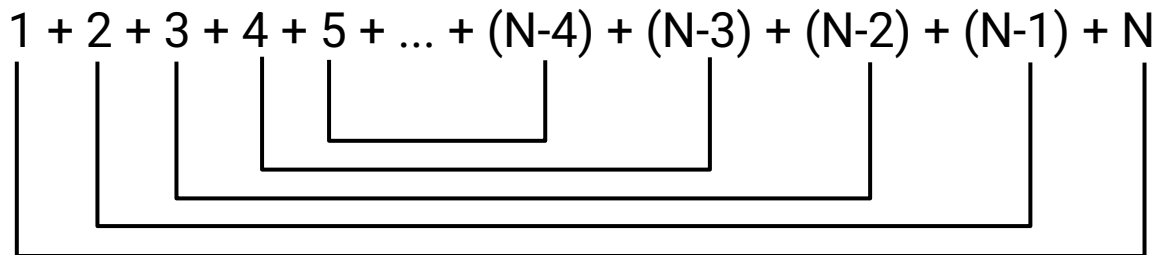
$N + 1$

$(N-1) + 2$

$(N-2) + 3$

...

There are  $N/2$  such pairs.



## SLList vs. AList Timing Tables

---

Below we see the time to insert N items for SLLists and ALists.

```
jug ~/.../speedtest  
$ java SpeedTestSLListTable
```

| N        | Time (ms) |
|----------|-----------|
| 10000    | 1         |
| 100000   | 3         |
| 1000000  | 71        |
| 10000000 | 875       |

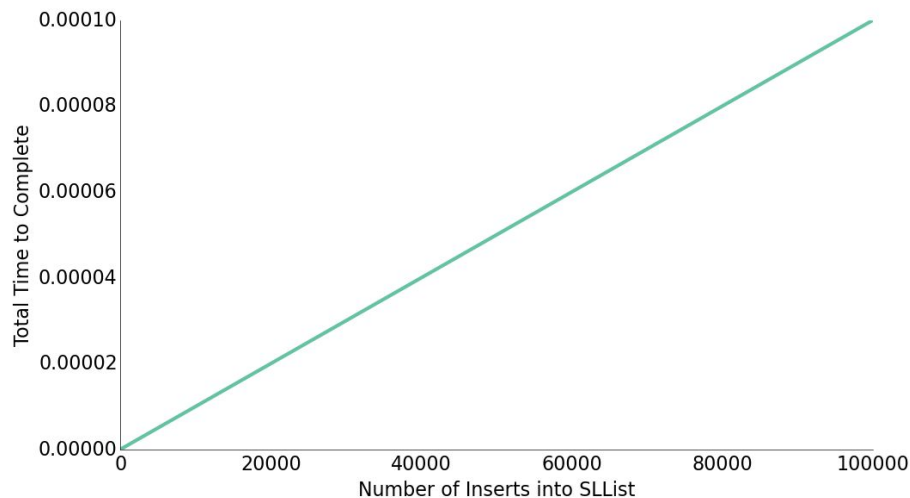
```
jug ~/.../speedtest  
$ java SpeedTestAListTable
```

| N       | Time (ms) |
|---------|-----------|
| 10000   | 19        |
| 100000  | 903       |
| 1000000 | 70227     |

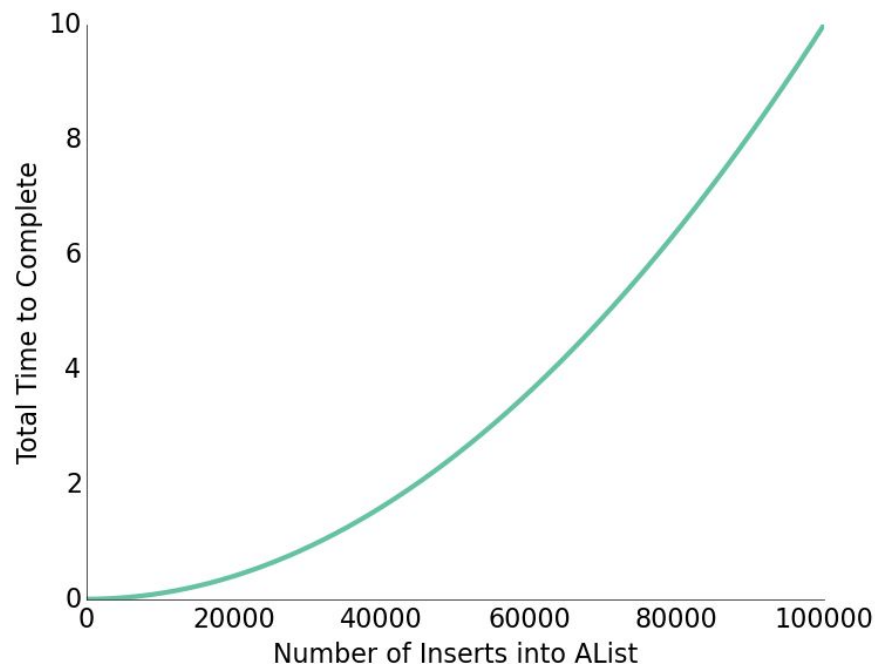
## Resizing Slowness (Theoretical Plots)

Inserting 100,000 items requires roughly 5,000,000,000 new containers.

- Computers operate at the speed of GHz (due billions of things per second).
- No huge surprise that 100,000 items took roughly seconds.



Note: Insert here is addFirst



## Resizing Slowness (Theoretical Plots)

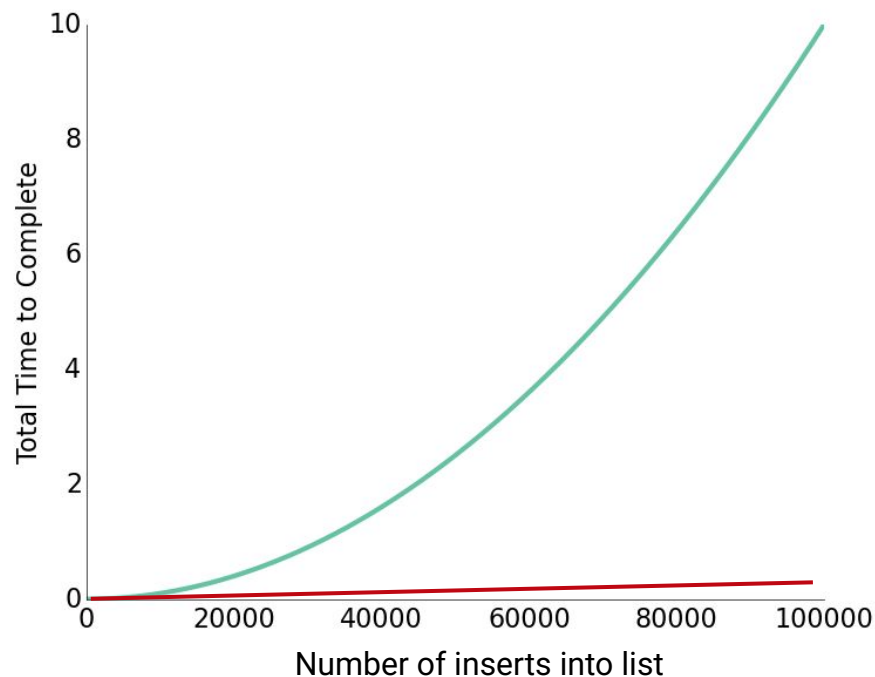
Inserting 100,000 items requires roughly 5,000,000,000 new containers.

- Computers operate at the speed of GHz (due billions of things per second).
- No huge surprise that 100,000 items took roughly seconds.

The same graphs from the previous slide, placed on top of each other.

Red line = SLList

Teal line = AList



# Better Resizing Strategy

---

Lecture 8, CS61B, Spring 2026

## Resizing ArrayLists

- Resizing Array Theory
- Resizing Array Implementation
- Runtime Analysis (Warmup)
- Runtime Analysis
- **Better Resizing Strategy**

Generic ArrayLists

Optimizing ArrayLists

- Linear Implementation of Circular Arrays

## Fixing the Resizing Performance Bug

How do we fix this?

```
public void resize(int capacity) {  
    int[] resized = new int[capacity];  
    for (int i = 0; i < size; i++) {  
        resized[i] = items[i];  
    }  
    items = resized;  
}  
  
public void addLast(int x) {  
    if (size == items.length) {  
        resize(size + 1);  
    }  
    items[size] = x;  
    size += 1;  
}
```

## Demo: Speed Testing the ArrayList

```
public void addLast(int x) {  
    if (size == items.length) {  
        resize(size + 1000);  
    }  
    items[size] = x;  
    size += 1;  
}
```

```
jug ~/.../speedtest  
$ java SpeedTestAList
```

Total runtime: 6 ms

N=100,000

Resizing by 1000 elements instead  
of 1 seems to speed up adding  
100,000 items... (was 602ms)

```
jug ~/.../speedtest  
$ java SpeedTestAList
```

Total runtime: 6097 ms

N=10,000,000

...but adding 10,000,000 items is  
still super slow.

## Demo: Speed Testing the ArrayList

```
public void addLast(int x) {  
    if (size == items.length) {  
        resize(size * 2);  
    }  
    items[size] = x;  
    size += 1;  
}
```

```
jug ~/.../speedtest  
$ java SpeedTestAList
```

Total runtime: 2 ms

N=100,000

If we double the array capacity  
every time it's full, then adding  
100,000 items is fast...

```
jug ~/.../speedtest  
$ java SpeedTestAList
```

Total runtime: 25 ms

N=10,000,000

...and adding 10,000,000 items is  
also fast.

## (Probably) Surprising Fact

Geometric resizing is much faster.

We can't prove this until later. [See this video for a more detailed analysis.](#)

Rough intuition: As the array grows larger, we resize less often.

```
public void addLast(int x) {  
    if (size == items.length) {  
        resize(size + RFACTOR);  
    }  
    items[size] = x;  
    size += 1;  
}
```

Great performance. 

This is how the Python list is implemented.

 Unusably bad.

```
public void addLast(int x) {  
    if (size == items.length) {  
        resize(size * RFACTOR);  
    }  
    items[size] = x;  
    size += 1;  
}
```

Suppose we have a very rare situation occur which causes us to:

- Insert 1,000,000,000 items.
- Then remove 990,000,000 items.

Our data structure will execute these operations acceptably fast, but afterwards there is a problem.

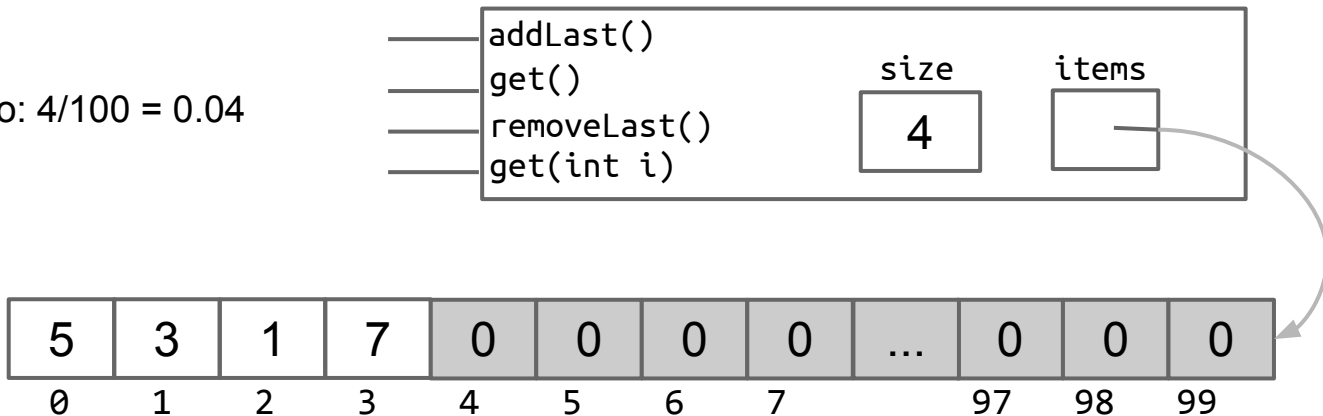
- What is the problem?

# Memory Efficiency

An AList should not only be efficient in time, but also efficient in space.

- Define the “usage ratio”  $R = \text{size} / \text{items.length}$ ;
- Typical solution: Half array size when  $R < 0.25$ .
- More details in a few weeks.

Usage ratio:  $4/100 = 0.04$



Later we will consider tradeoffs between time and space efficiency for a variety of algorithms and data structures.

# AList Problems to Address

```
public class AList {
    private int[] items;
    private int size;

    public AList() {
        items = new int[999];
        size = 0;
    }

    public void addLast(int x) {
        items[size] = x;
        size += 1;
    }

    public int get(int i) {
        return items[i];
    }

    public int size() {
        return size;
    }
}
```

AList problems to address today:

- ~~Expanding our array to support unlimited elements~~
- Generics: making AList able to store anything
- Optimizing AList speed

## AList Improvements

|    |                                           |
|----|-------------------------------------------|
| #1 | Fast to access items anywhere in the list |
| #2 | Allow re-sizing of the underlying array   |

# Generic ArrayLists

---

Lecture 8, CS61B, Spring 2026

## Resizing ArrayLists

- Resizing Array Theory
- Resizing Array Implementation
- Runtime Analysis (Warmup)
- Runtime Analysis
- Better Resizing Strategy

## Generic ArrayLists

## Optimizing ArrayLists

- Linear Implementation of Circular Arrays

```
public class AList {
    private int[] items;
    private int size;

    public AList() {
        items = new int[8];
        size = 0;
    }

    public void resize(int capacity) {
        int[] resized = new int[capacity];
        for (int i = 0; i < size; i++) {
            resized[i] = items[i];
        }
        items = resized;
    }

    public int get(int i) {
        return items[i];
    }

    ...
}
```

What would we change to make it generic?



## Generic ALists (similar to generic SLists): [yellkey.com/spring](http://yellkey.com/spring)

```
public class AList {
    private int[] items;
    private int size;

    public AList() {
        items = new int[8];
        size = 0;
    }

    public void resize(int capacity) {
        int[] resized = new int[capacity];
        for (int i = 0; i < size; i++) {
            resized[i] = items[i];
        }
        items = resized;
    }

    public int get(int i) {
        return items[i];
    }
    ...
}
```

```
public class AList<Glorp> {
    private Glorp[] items;
    private int size;

    public AList() {
        items = (Glorp[]) new Object[8];
        size = 0;
    }

    public void resize(int cap) {
        int[] resized = (Glorp[]) new Object[cap];
        for (int i = 0; i < size; i++) {
            resized[i] = items[i];
        }
        items = resized;
    }

    public Glorp get(int i) {
        return items[i];
    }
    ...
}
```

## Generic ALists (similar to generic SLists)

```
public class AList<Glorp> {
    private Glorp[] items;
    private int size;

    public AList() {
        items = (Glorp[]) new Object[8];
        size = 0;
    }

    public void resize(int cap) {
        int[] resized = (Glorp[]) new Object[cap];
        for (int i = 0; i < size; i++) {
            resized[i] = items[i];
        }
        items = resized;
    }

    public Glorp get(int i) {
        return items[i];
    }
    ...
}
```

When creating an array of references to Glorps:

- `(Glorp[]) new Object[8];`
- Causes a compiler warning, which you should ignore.

Why not just `new Glorp[cap];`

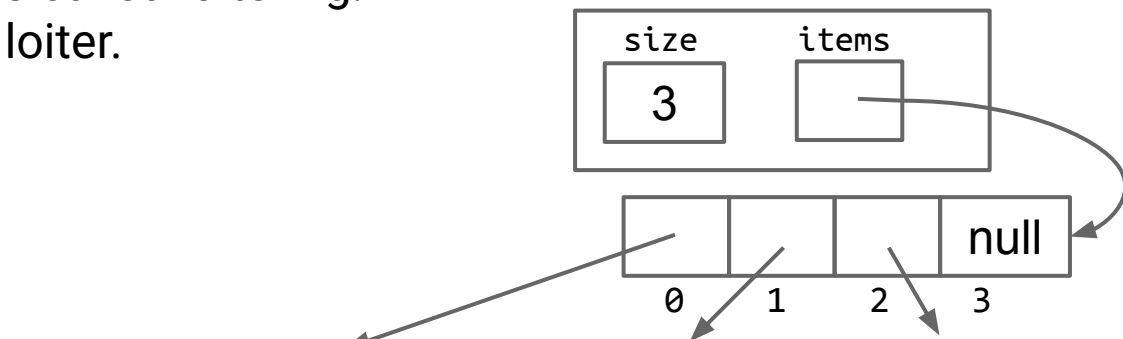
- Will cause a “generic array creation” error.

## Nulling Out Deleted Items

Unlike integer based ALists, we actually want to null out deleted items.

- Java only destroys unwanted objects when the last reference has been lost.
- Keeping references to unneeded objects is sometimes called loitering.
- Save memory. Don't loiter.

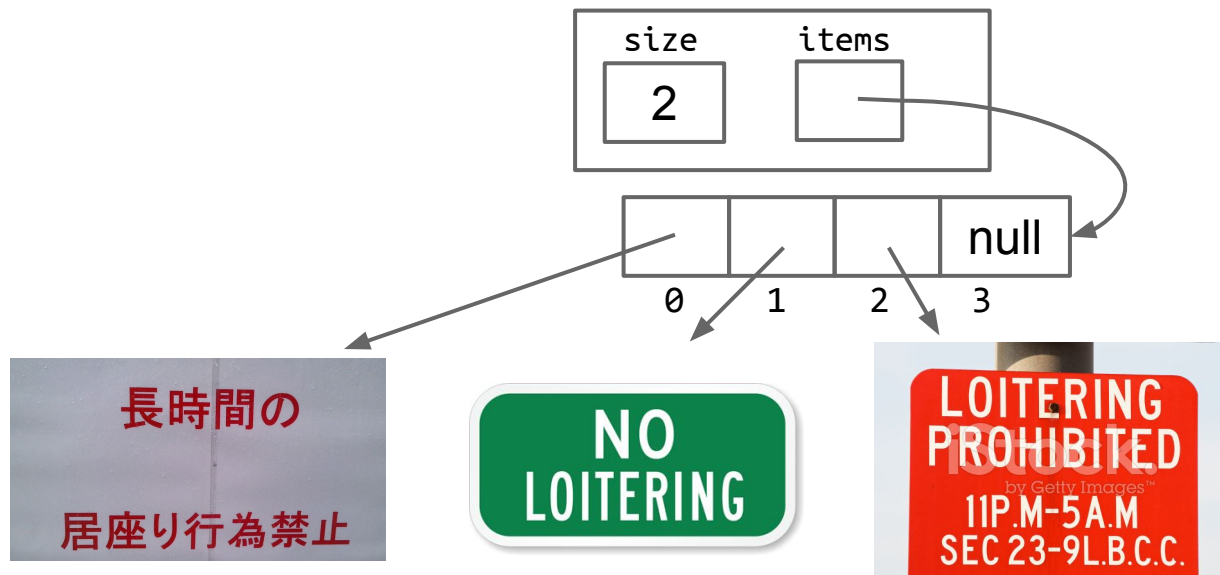
```
public Glorp removeLast() {  
    Glorp returnItem = getLast();  
    items[size - 1] = null;  
    size -= 1;  
    return returnItem;  
}
```



## Loitering Example

Changing size to 2 yields a correct AList.

- But memory is wasted storing a reference to the red sign image.

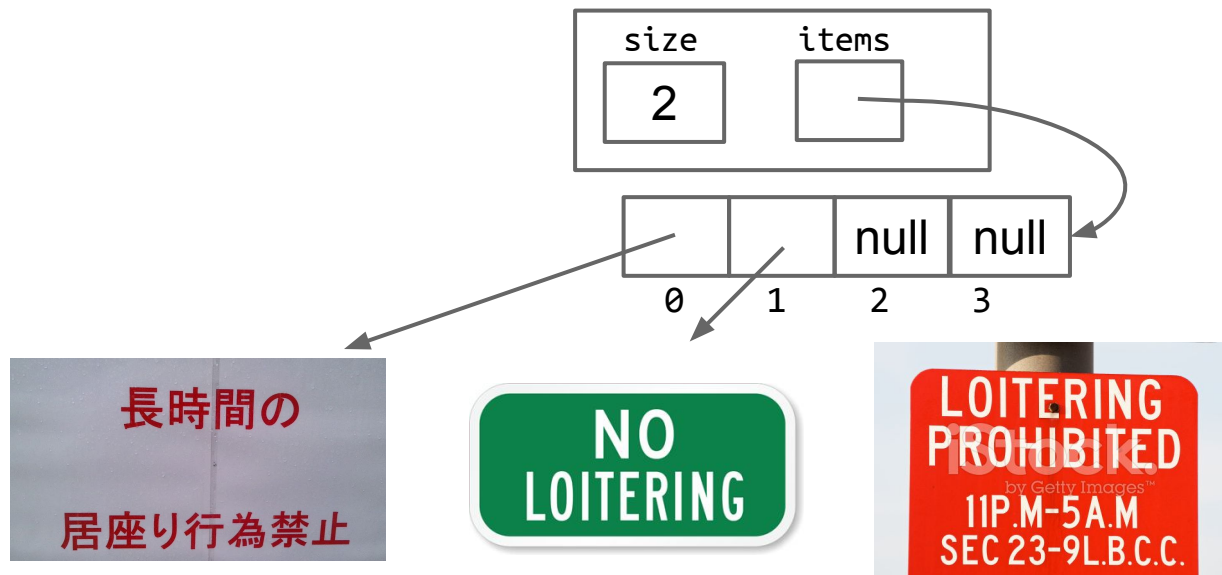


## Loitering Example

Changing size to 2 yields a correct AList.

- But memory is wasted storing a reference to the red sign image.

By nulling out items[2], Java is free to destroy the unneeded image from memory, which could be potentially megabytes in size.

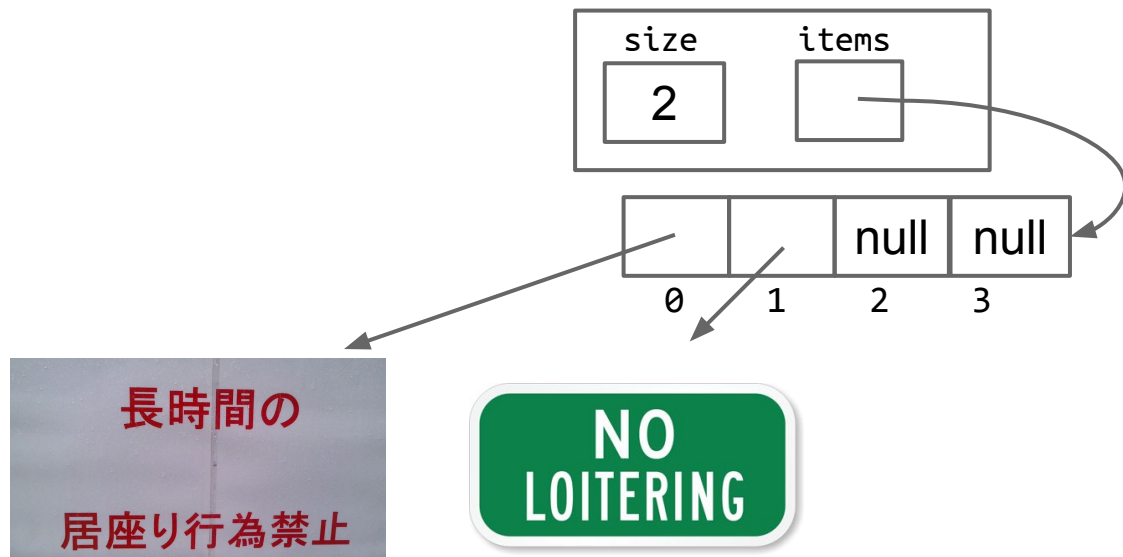


## Loitering Example

Changing size to 2 yields a correct AList.

- But memory is wasted storing a reference to the red sign image.

By nulling out items[2], Java is free to destroy the unneeded image from memory, which could be potentially megabytes in size.



# AList Problems to Address

```
public class AList<Glorp> {  
    private Glorp[] items;  
    private int size;  
  
    public AList() {  
        items = (Glorp[]) new Object[8];  
        size = 0;  
    }  
  
    public void resize(int cap) {  
        int[] resized = (Glorp[]) new Object[cap];  
        for (int i = 0; i < size; i++) {  
            resized[i] = items[i];  
        }  
        items = resized;  
    }  
  
    public Glorp get(int i) {  
        return items[i];  
    }  
    ...  
}
```

AList problems to address today:

- ~~Expanding our array to support unlimited elements~~
- ~~Generics: making AList able to store anything~~
- Optimizing AList speed

## AList Improvements

|    |                                           |
|----|-------------------------------------------|
| #1 | Fast to access items anywhere in the list |
| #2 | Allow re-sizing of the underlying array   |
| #3 | Generics: AList can store anything        |

# Optimizing ArrayLists

---

Lecture 8, CS61B, Spring 2026

## Resizing ArrayLists

- Resizing Array Theory
- Resizing Array Implementation
- Runtime Analysis (Warmup)
- Runtime Analysis
- Better Resizing Strategy

## Generic ArrayLists

### **Optimizing ArrayLists**

- Linear Implementation of Circular Arrays

## What are ALists slow at?

```
public class AList<Glorp> {
    private Glorp[] items;
    private int size;

    public AList() {
        items = (Glorp[]) new Object[8];
        size = 0;
    }

    private void resize(int cap) {
        Glorp[] a = (Glorp[]) new Object[cap];
        System.arraycopy(items, 0,
            a, 0, size);
        items = a;
    }

    public Glorp get(int i) {
        return items[i];
    }
    ...
}
```

We build AList to make get(i) fast.

Was there anything better about DLList?

```
public class DLList<Pizza> {
    private Node sentinel;
    private int size;

    public class Node {
        public Pizza item;
        public Node prev;
        public Node next;
        ...
    }

    ...
}
```

We built AList to make `get(i)` fast.

Are ArrayLists strictly better? Is there anything a LinkedList is better at?

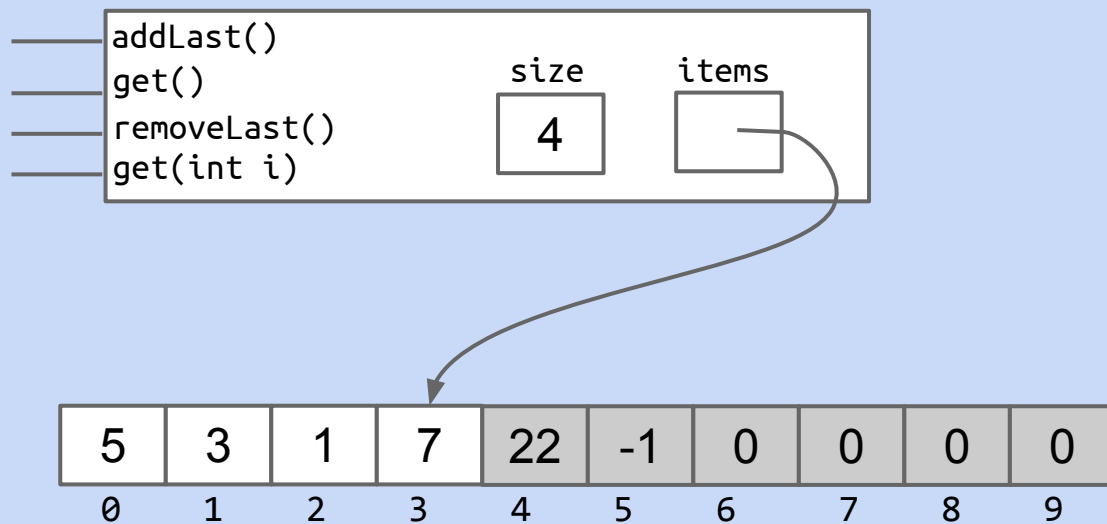
- Yes, `addFirst` and `removeFirst` are faster for linked lists.
- Less important: LinkedList operations at ends are always fast at the ends. ArrayList operations sometimes cause a linear time resize (but negligible in practice).

## Speeding up Front Operations

For our AList class, `addFirst` or `removeFirst` are going to be slow.

- Have to shift everything to the left.

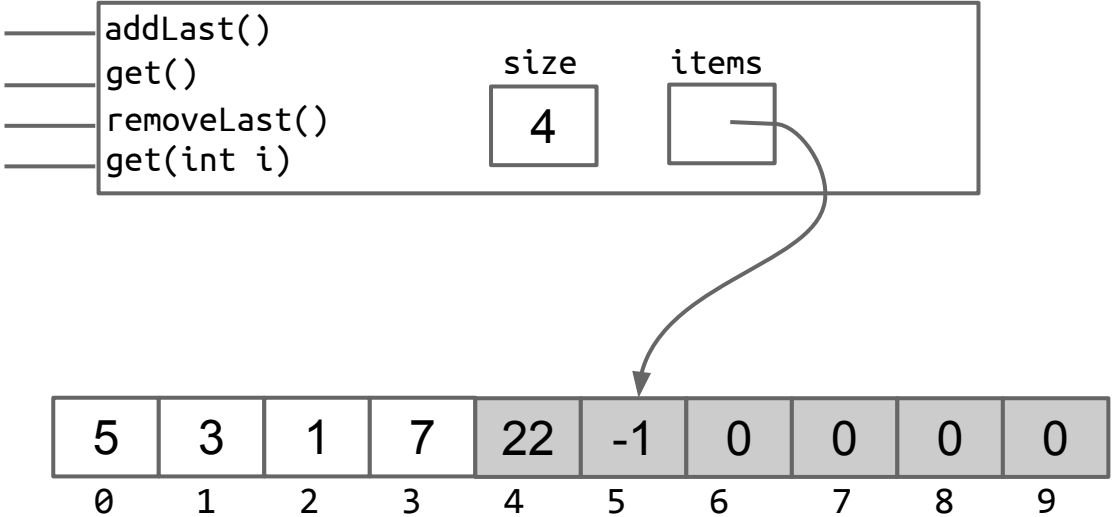
How might we modify our design so that these front operations are fast?



# Speeding up Front Operations

[5, 3, 1, 7]

User's Mental Model

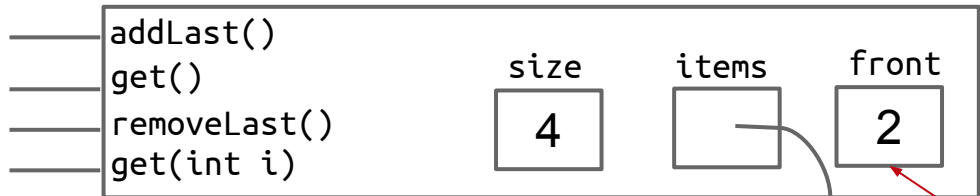


# Speeding up Front Operations

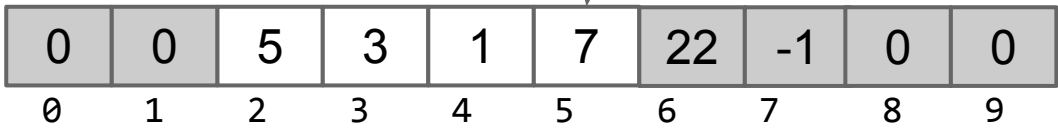
What if we moved everything in our list over?

[5, 3, 1, 7]

User's Mental Model



Now we need to keep track of where the list starts!



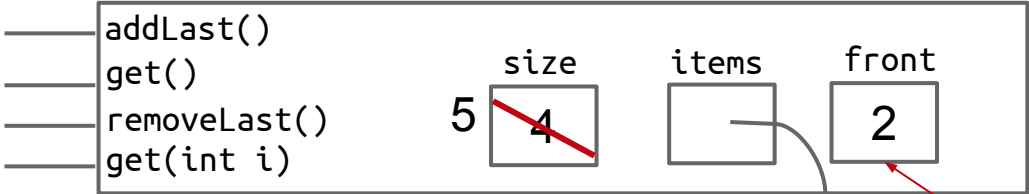
# Speeding up Front Operations

What if we moved everything in our list over?

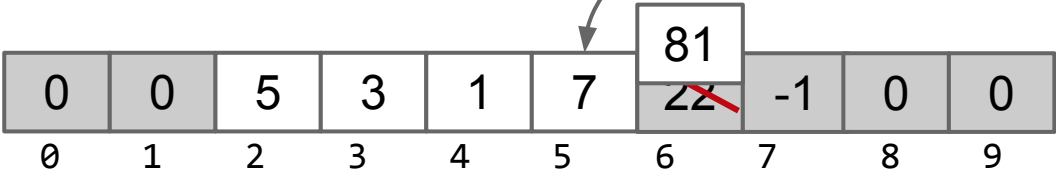
```
addLast(81);
```

[5, 3, 1, 7, 81]

User's Mental Model



Now we need to keep track of where the list starts!



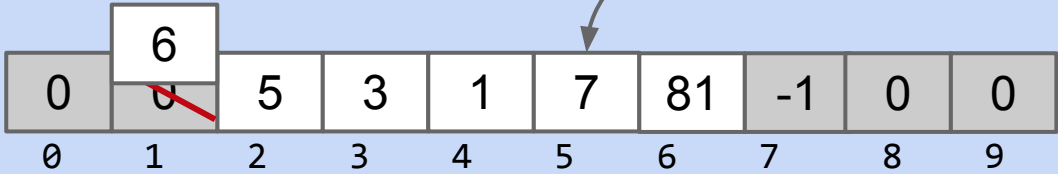
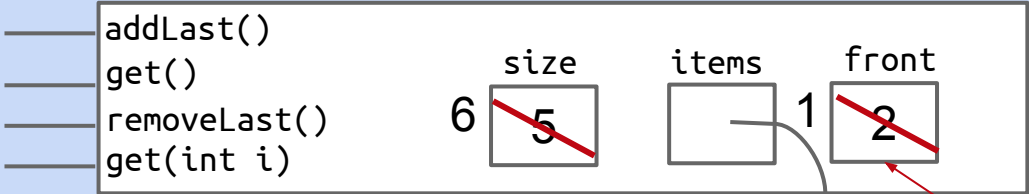
# Speeding up Front Operations

What if we moved everything in our list over?

```
addLast(81);  
addFirst(6);
```

[6, 5, 3, 1, 7, 81]

User's Mental Model



Now we need to keep track of where the list starts!

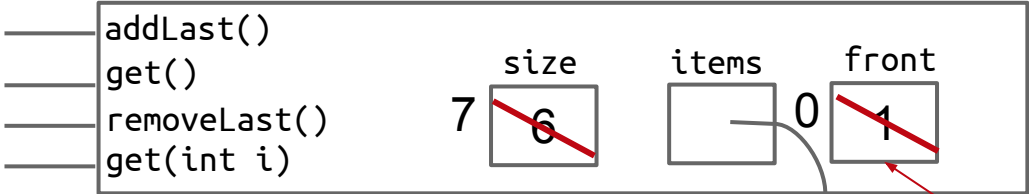
# Speeding up Front Operations

What if we moved everything in our list over?

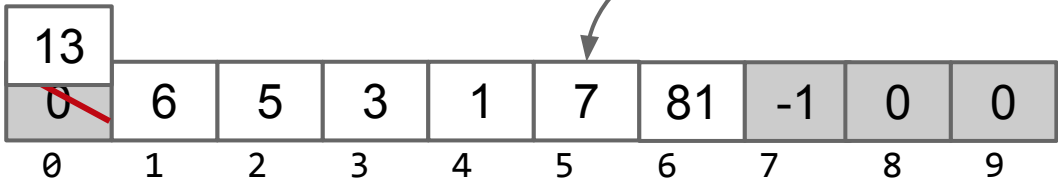
```
addLast(81);  
addFirst(6);  
addFirst(13);
```

[13, 6, 5, 3, 1, 7, 81]

User's Mental Model



Now we need to keep track of where the list starts!



# Speeding up Front Operations

What if we moved everything in our list over?

addLast(81); [11, 13, 6, 5, 3, 1, 7, 81]

addFirst(6); User's Mental Model

addFirst(13);

addLast()  
get()  
removeLast()  
get(int i)

size

7

items

front

0



Where can we put the 11?



# Speeding up Front Operations

What if we moved everything in our list over?

addLast(81); **[11, 13, 6, 5, 3, 1, 7, 81]**

addFirst(6); User's Mental Model

addFirst(13);

addFirst(11);

addLast()  
get()  
removeLast()  
get(int i)

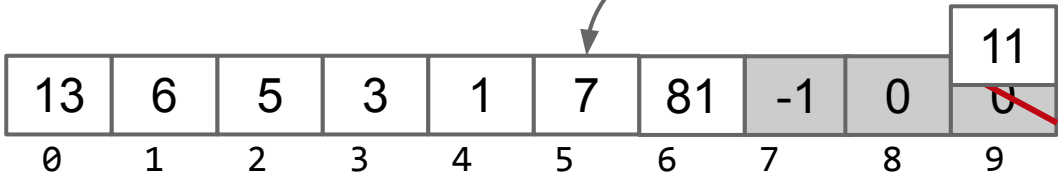
size  
8 ~~7~~

items  
[ ]

front  
9 ~~0~~



Where can we put the 11?



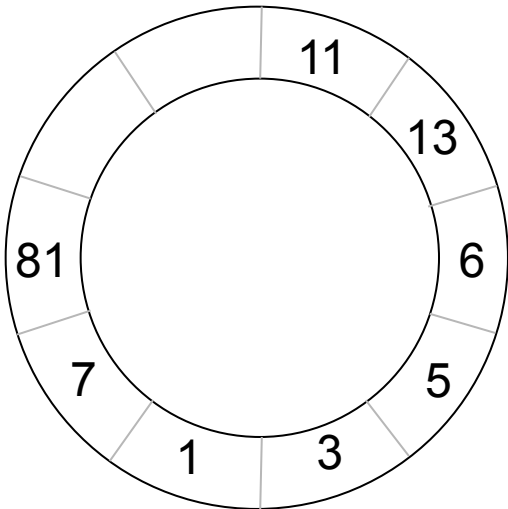
# Test Your Understanding

Is the representation unique?

No!

[11, 13, 6, 5, 3, 1, 7, 81]

User's Mental Model



Conceptual Circular Array

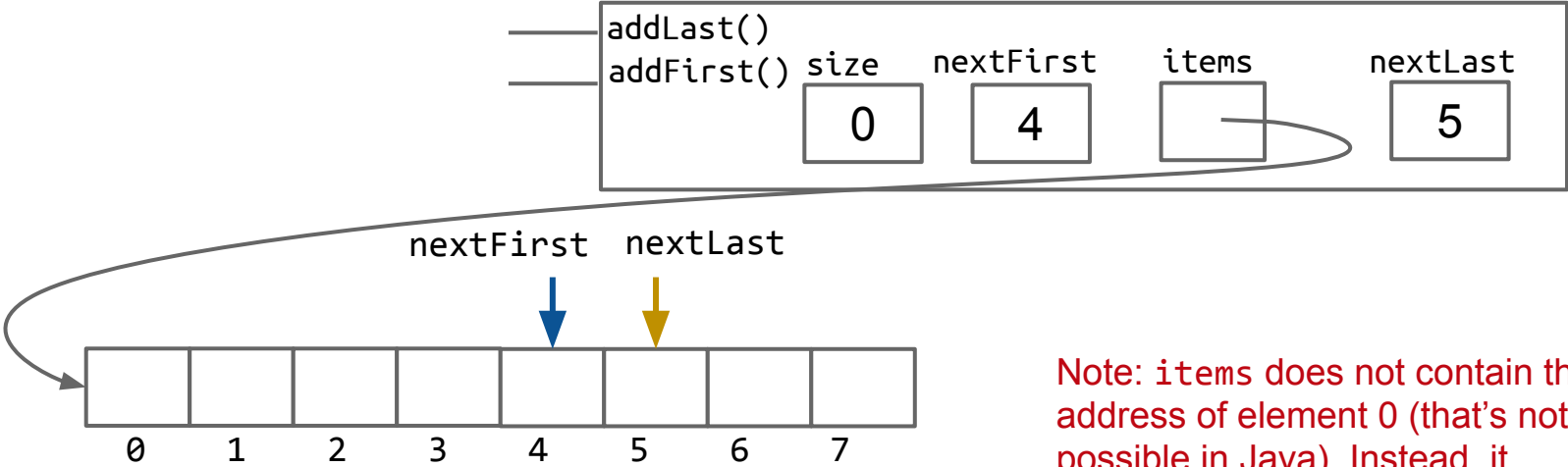
|    |   |   |   |   |   |    |   |   |    |
|----|---|---|---|---|---|----|---|---|----|
| 13 | 6 | 5 | 3 | 1 | 7 | 81 |   |   | 11 |
| 0  | 1 | 2 | 3 | 4 | 5 | 6  | 7 | 8 | 9  |

|    |    |   |   |   |   |   |    |   |   |
|----|----|---|---|---|---|---|----|---|---|
| 11 | 13 | 6 | 5 | 3 | 1 | 7 | 81 |   |   |
| 0  | 1  | 2 | 3 | 4 | 5 | 6 | 7  | 8 | 9 |

|   |   |    |   |   |    |    |   |   |   |
|---|---|----|---|---|----|----|---|---|---|
| 1 | 7 | 81 |   |   | 11 | 13 | 6 | 5 | 3 |
| 0 | 1 | 2  | 3 | 4 | 5  | 6  | 7 | 8 | 9 |

Starting from an empty ArrayList:

Conceptual List: []



Note: `items` does not contain the address of element 0 (that's not possible in Java). Instead, it contains the address of the entire array!

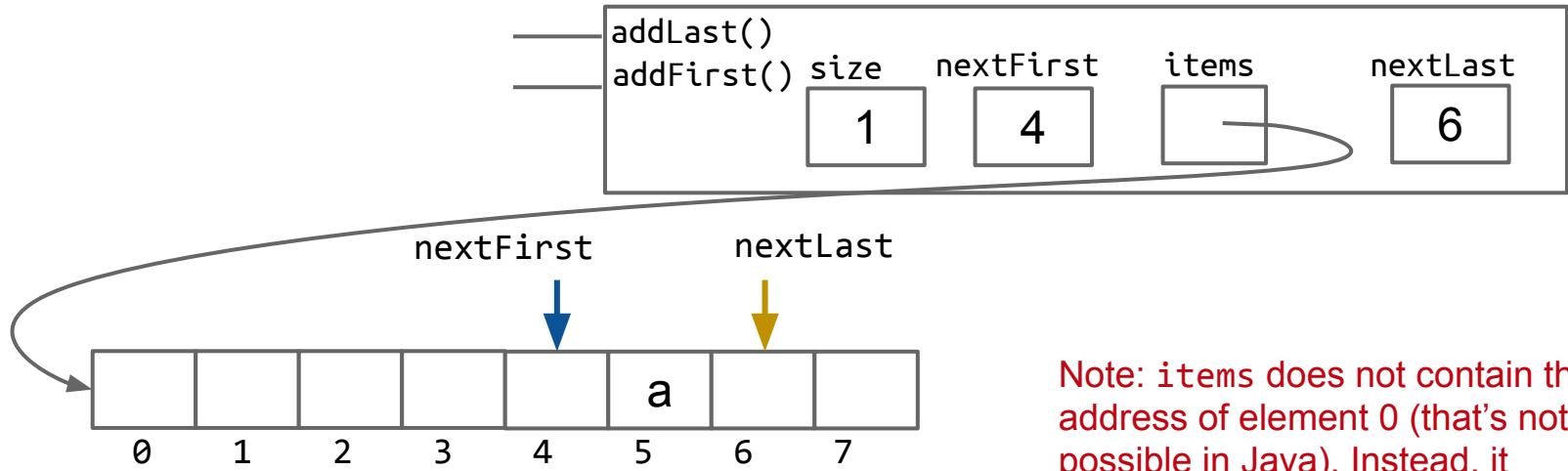
You don't have to start your array at 4 or 5. I just picked these arbitrarily.

# Linear Array Demo

Starting from an empty ArrayList:

- `addLast("a")`

Conceptual List: [a]



Note: `items` does not contain the address of element 0 (that's not possible in Java). Instead, it contains the address of the entire array!

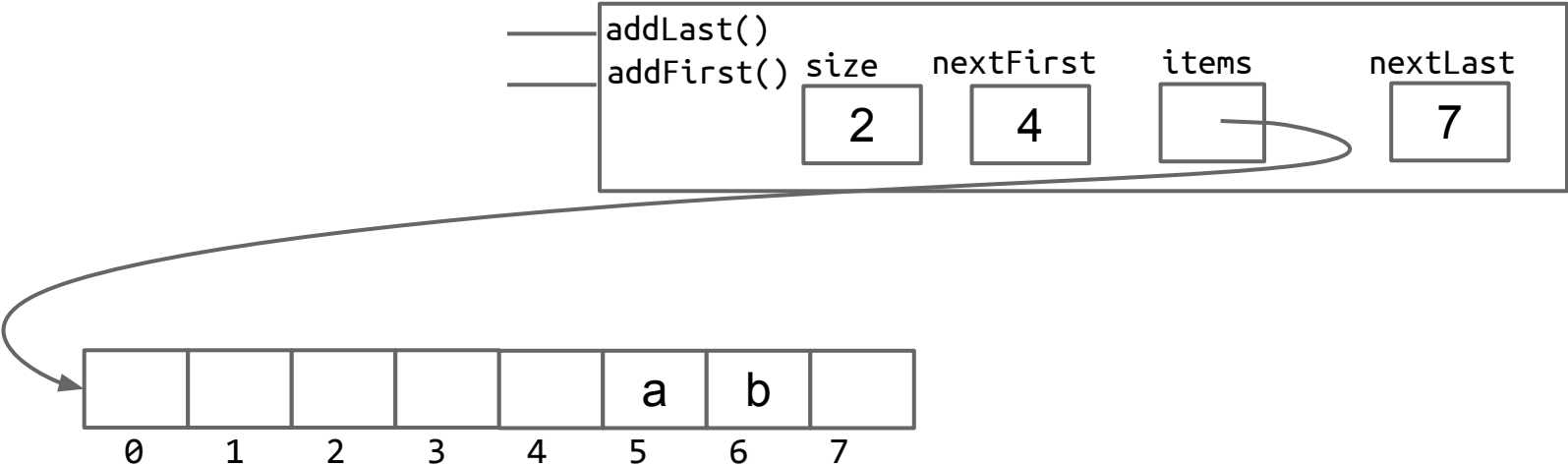
You don't have to start your array at 4 or 5. I just picked these arbitrarily.

# Linear Array Demo

Starting from an empty ArrayList:

- addLast(“a”)
- addLast(“b”)

Conceptual List: [a, b]



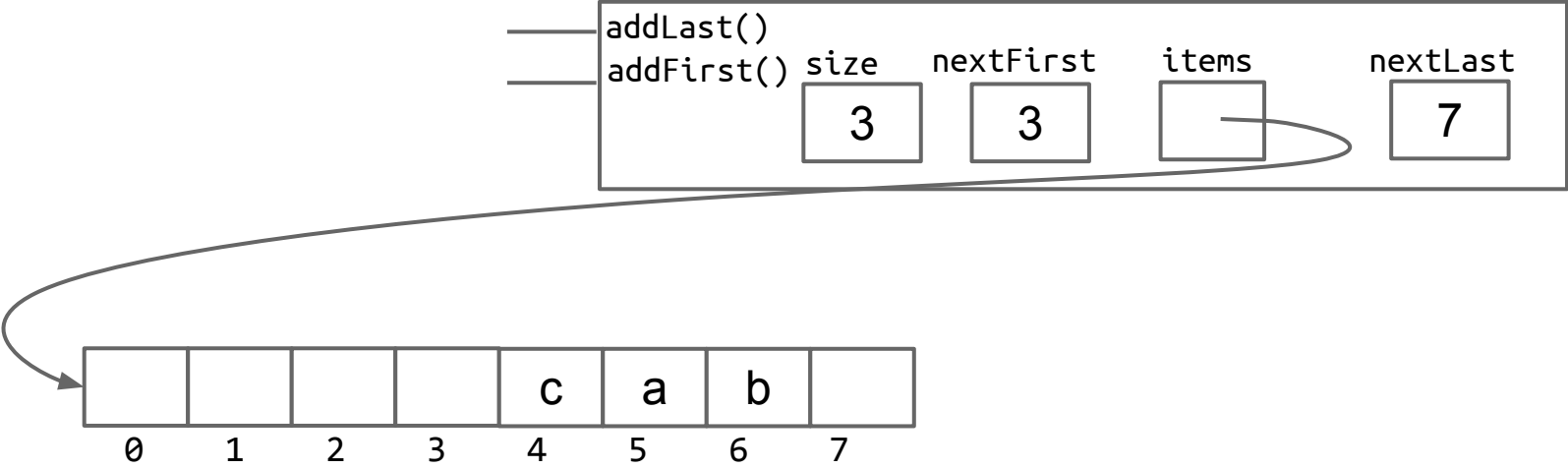
You don't have to start your array at 4 or 5. I just picked these arbitrarily.

# Linear Array Demo

Starting from an empty ArrayList:

- addLast("a")
- addLast("b")
- addFirst("c")

Conceptual List: [c, a, b]



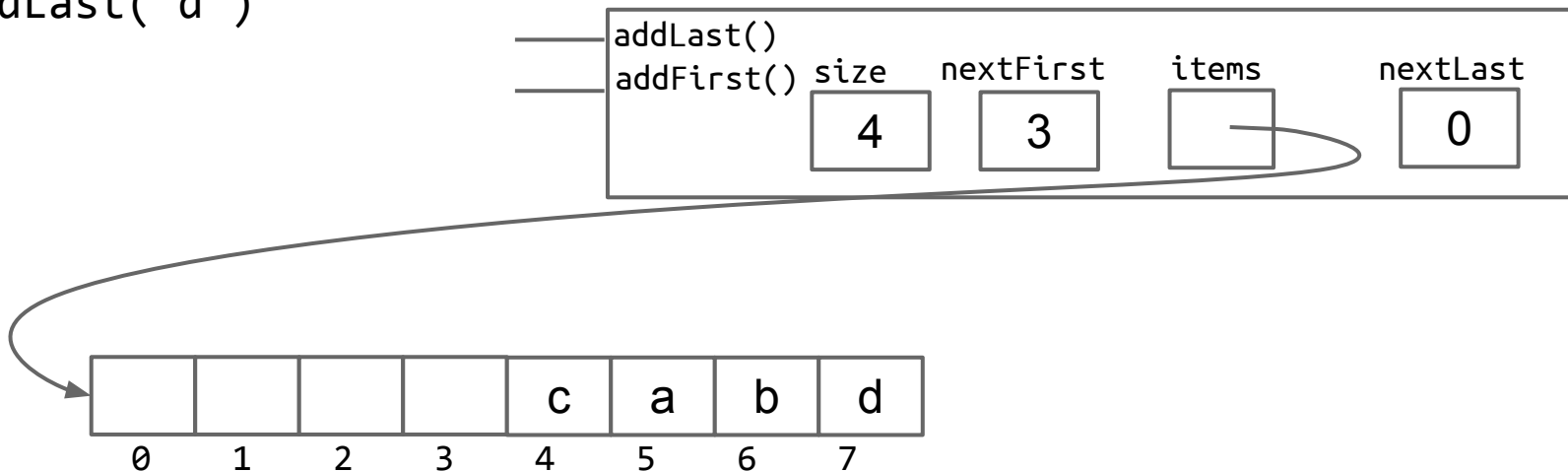
You don't have to start your array at 4 or 5. I just picked these arbitrarily.

# Linear Array Demo

Starting from an empty ArrayList:

- `addLast("a")`
- `addLast("b")`
- `addFirst("c")`
- `addLast("d")`

Conceptual List: [c, a, b, d]



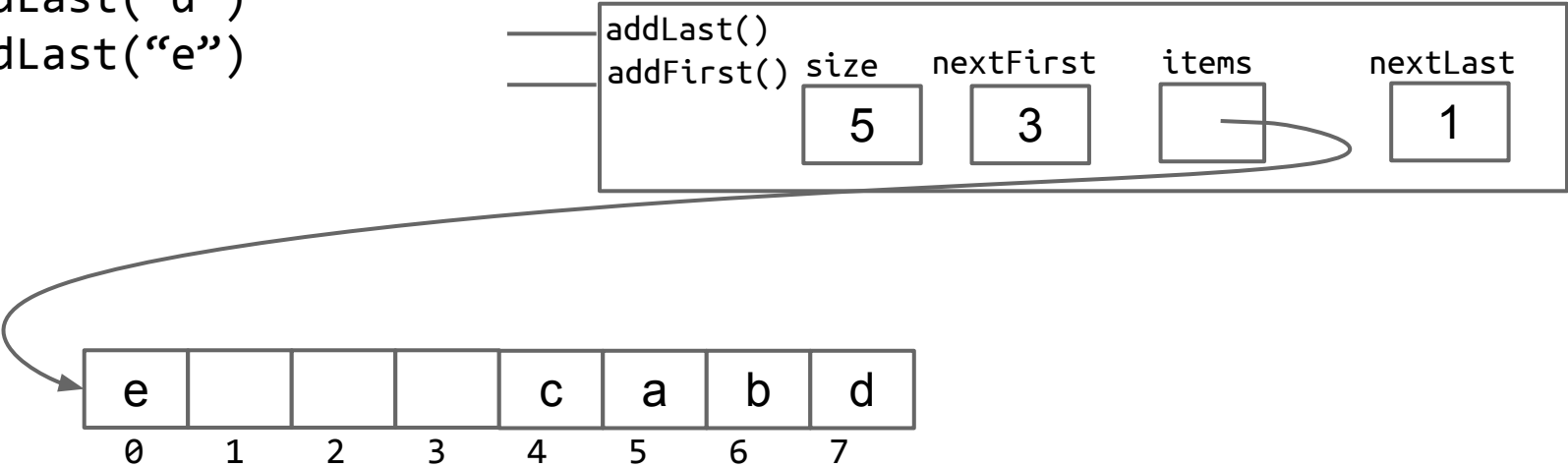
You don't have to start your array at 4 or 5. I just picked these arbitrarily.

# Linear Array Demo

Starting from an empty ArrayList:

- addLast("a")
- addLast("b")
- addFirst("c")
- addLast("d")
- addLast("e")

Conceptual List: [c, a, b, d, e]



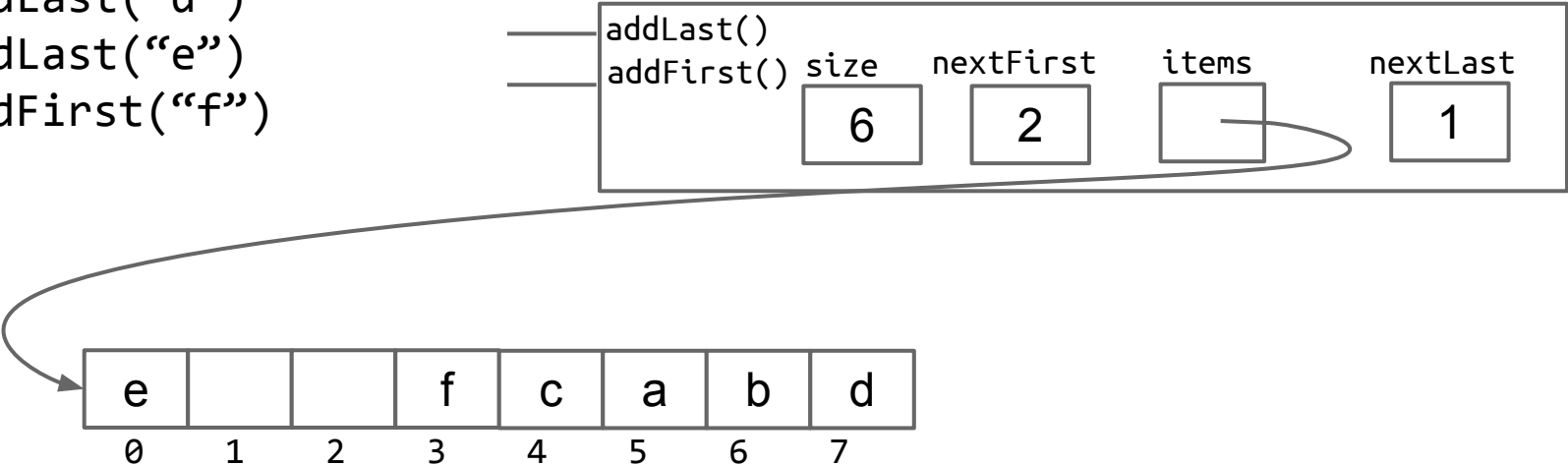
You don't have to start your array at 4 or 5. I just picked these arbitrarily.

# Linear Array Demo

Starting from an empty ArrayList:

- addLast("a")
- addLast("b")
- addFirst("c")
- addLast("d")
- addLast("e")
- addFirst("f")

Conceptual List: [f, c, a, b, d, e]



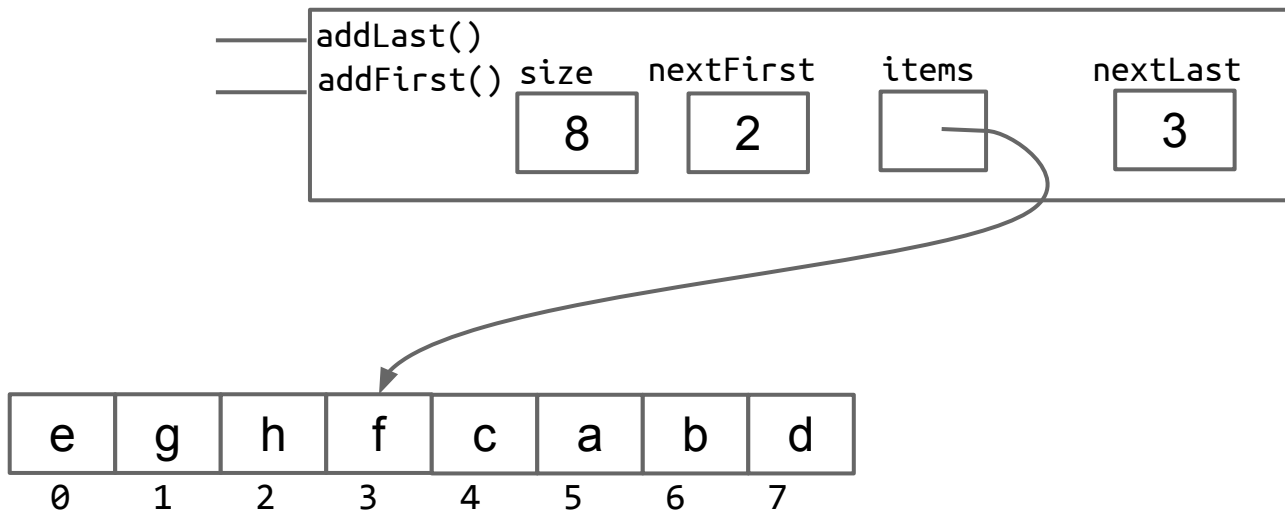
You don't have to start your array at 4 or 5. I just picked these arbitrarily.

## Linear Array Demo

Starting from an empty ArrayList:

- `addLast("a")`
- `addLast("b")`
- `addFirst("c")`
- `addLast("d")`
- `addLast("e")`
- `addFirst("f")`
- `addLast("g")`
- `addLast("h")`

Conceptual List: [f, c, a, b, d, e, g, h]



After the next **addLast** operation, we'll need a bigger array.

# Resizing: You Can Pick Whatever Representation You Prefer for Project 2

Starting from an empty ArrayList:

- `addLast("a")`
- `addLast("b")`
- `addFirst("c")`
- `addLast("d")`
- `addLast("e")`
- `addFirst("f")`
- `addLast("g")`
- `addLast("h")`
- **`addLast("Z")`**

Conceptual List: [f, c, a, b, d, e, g, h, Z]

Before **`addLast("Z")`**:

|   |   |   |   |   |   |   |   |
|---|---|---|---|---|---|---|---|
| e | g | h | f | c | a | b | d |
| 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 |

After **`addLast("Z")`**: (all of the below are valid choices)

|   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| e | g | h | Z |   |   |   |   |   |   |   | f | c | a | b | d |
| f | c | a | b | d | e | g | h | Z |   |   |   |   |   |   |   |
|   |   |   |   | f | c | a | b | d | e | g | h | Z |   |   |   |

It's your choice what array to use, e.g. all of the three choices above are valid!

Take things a little at a time.

- Writing tons of code all at once is going to lead to misery and only misery.
- If you wrote too much stuff and feel overwhelmed, comment out whatever is unnecessary.

Consider not doing resizing at all until you know your code works without it.

- Resizing is a performance optimization (and is required for full credit).
- Recommendation: Code that uses `arraycopy` can be hard to get right.
- Observation: You might find the `get` method useful in your `resize` method.

If your first try goes badly, don't be afraid to scrap your code and start over.

- There isn't that much code.

# AList Problems to Address

```
public class AList<Glorp> {  
    private Glorp[] items;  
    private int size;  
  
    public AList() {  
        items = (Glorp[]) new Object[8];  
        size = 0;  
    }  
  
    public void resize(int cap) {  
        int[] resized = (Glorp[]) new Object[cap];  
        for (int i = 0; i < size; i++) {  
            resized[i] = items[i];  
        }  
        items = resized;  
    }  
  
    public Glorp get(int i) {  
        return items[i];  
    }  
    ...  
}
```

AList problems to address today:

- Expanding our array to support unlimited elements
- Generics: making AList able to store anything
- Optimizing AList speed

## AList Improvements

|    |                                           |
|----|-------------------------------------------|
| #1 | Fast to access items anywhere in the list |
| #2 | Allow re-sizing of the underlying array   |
| #3 | Generics: AList can store anything        |
| #4 | Make list circular                        |

Python lists do NOT use this circular abstraction.

- `x.insert(0, "hi")` is slow for Python lists.

Why do you think this is?

Python lists do NOT use this circular abstraction.

- `x.insert(0, "hi")` is slow for Python lists.

Why do you think this is?

- Insertion at the front of a list is too rare to justify increased code complexity and performance penalty for other operations.
  - Example: e.g. `x[3:9]` might have `x[3:6]` at the end of the underlying array, and `x[6:9]` at the beginning.

## Selecting the Right Data Structure

---

Users who want fast front operations should use the appropriate data structure.

- `LinkedList` instead of `ArrayList` (in Java).
- `collections.deque` instead of a `list` in Python.
  - Variant of a list with fast front and back operations.

Your project 1 and project 2 solutions are required to have fast front operations (and back operations).

- For your `ArrayDeque`, you are required to use this circular approach.

(Fun (?) fact: Python's built-in `Deque` class is built using a linked list, not a `Deque`. For more see [https://stackoverflow.com/questions/45134139/why-is-deque-implemented-as-a-linked-list-instead-of-a-circular-array?utm\\_source=chatgpt.com](https://stackoverflow.com/questions/45134139/why-is-deque-implemented-as-a-linked-list-instead-of-a-circular-array?utm_source=chatgpt.com))

Attendance link: <https://www.yellkey.com/likely>

To-do:

- Project 1 due Tuesday.
- Get started on Project 2, due February 17.

